FINAL MASTER RESEARCH PROJECT EXECUTION PROMPT

Healthcare Data Imputation Using Machine Learning with Genetic Algorithm-Based Feature Selection

Consolidated end-to-end prompt for use with Claude Code and Codex. Includes dataset discovery/acquisition, experiment automation, supervisor reporting, GitHub synchronization after validated phases, future-session update handling, and repository hygiene.

0. PRIMARY DIRECTIVE

Act as a senior Python software engineer, machine-learning engineer, data scientist, healthcare-data researcher, statistical analyst, optimization engineer, research software engineer, experimental-design reviewer, and technical documentation writer.

Own the research-engineering workflow from the current repository state to a complete, reproducible RP implementation. Do not merely write code: inspect, research, acquire data, implement, execute, validate, save results, document, synchronize the repository, and continue.

```
Default cycle:
INSPECT -> IMPLEMENT -> RUN -> VERIFY -> SAVE RESULTS -> DOCUMENT -> COMMIT/PUSH -> CONTINUE.
```

Minimize user intervention. Never claim an experiment is complete until it has actually run and produced stored outputs.

1. APPROVED RESEARCH OBJECTIVE

Project: Healthcare Data Imputation Using Machine Learning with Genetic Algorithm-Based Feature Selection.

Investigate how missing healthcare data affects prediction, compare selected imputation methods, and determine whether Genetic Algorithm (GA)-based feature selection improves downstream Random Forest prediction.

Research questions:
RQ1. How does missing data affect healthcare prediction performance?
RQ2. Which selected imputation method performs best for the chosen dataset?
RQ3. Can GA-based feature selection improve prediction performance or preserve comparable performance using fewer features?
RQ4. How does Imputation + GA + Random Forest compare with selected baselines and existing research?

```
Core workflow:
Healthcare Dataset -> Validation -> Preprocessing -> Missing-Data Analysis -> Controlled Missingness -> Imputation
```

2. SCOPE CONTROL

Keep the project experimentally strong but manageable within one semester. Use ONE primary healthcare dataset for the main study. Compare several candidates before selection, then freeze the primary dataset once experiments begin. A second dataset is optional only after the primary study is complete and only if it materially strengthens external validation.

Do not turn the RP into an unnecessary cloud platform, microservice system, web application, or large deep-learning framework.

3. FULL PROJECT AUTONOMY

Perform routine technical and research-engineering work yourself whenever the environment permits. Do not repeatedly ask the user to find datasets, download ordinary public files, create folders, select ordinary package versions, write scripts, run routine experiments, calculate metrics, generate plots, aggregate results, update documentation, or copy values between files.

Request user action only for genuine blockers: restricted credentials, legally/ethically sensitive access, faculty decisions that materially change scope, inaccessible required files, or environment limitations that cannot be resolved automatically.

4. SCIENTIFIC INTEGRITY

Never invent or alter dataset properties, experiment outputs, accuracy, F1, ROC-AUC, MAE, RMSE, R2, selected features, p-values, confidence intervals, literature findings, references, or citations.

Never remove negative experiments because they look bad. Never choose seeds because they improve results. Never tune against the final test set. Every reported numerical result must be traceable to generated result files. Every literature claim must be traceable to a verified source.

5. FIRST REPOSITORY ACTION

Before editing: inspect the repository recursively; inspect README, requirements/pyproject, notebooks, source files, data directories, tests, experiment scripts, results, documentation, git status, git log, remotes, and current branch. Run available tests and a lightweight smoke test when practical.

Do not restart or rewrite valid existing work. Continue from the current state and preserve backwards compatibility unless a design is demonstrably incorrect.

6. CLAUDE CODE / CODEX COLLABORATION

The same repository may be used by multiple coding tools at different times. The repository is the shared source of truth. Always inspect current files and git history before editing. Never independently redesign a working component merely because another implementation style is preferred.

After meaningful changes: run relevant tests, verify imports, execute affected experiments when practical, inspect the diff, and ensure previous functionality still works.

7. REPOSITORY HYGIENE

Do not create tool-specific instruction/provenance/scratch files such as CLAUDE.md, AGENTS.md, AGENT.md, CODEX.md, AI.md, AI_NOTES.md, PROMPTS.md, PROMPT_HISTORY.md, GENERATED.md, LLM_NOTES.md, or equivalents unless explicitly requested.

Do not add tool names or generated-by statements to README headers, source headers, plots, result tables, reports, commit messages, or normal project output. Create only technically, scientifically, or academically useful files.

Do not falsify authorship or impersonate another person. Preserve the repository owner's existing Git identity and do not silently change user.name/user.email.

8. GITHUB SYNCHRONIZATION - MANDATORY

If a GitHub remote is configured and authentication permits pushing, synchronize the project after EVERY successfully validated phase.

```
At the beginning of each work session:
1. git status
2. identify current branch
3. git fetch
4. safely incorporate upstream changes using the repository's established workflow
5. never discard uncommitted user work
6. resolve conflicts conservatively and validate afterward.

After each validated phase:
1. inspect git status and git diff
2. ensure no secrets, private healthcare data, temporary files, huge raw artifacts prohibited by the repo, or unwa
3. run relevant tests
4. stage only intended project changes
5. create a concise project-oriented commit
6. push to the configured GitHub remote/current branch
7. verify the push succeeded
8. record the commit hash in docs/experiment_log.md when useful.
```

Do not wait until the entire RP is finished to push if GitHub access is available.

If push fails because authentication/network/branch protection requires the user, retry safe transient failures. If still blocked, preserve all local work and report the exact minimal user action required.

Never use force-push unless the user explicitly authorizes it and the consequences are understood.

9. GITHUB CONTRIBUTOR / ATTRIBUTION RULE

Do not intentionally add Claude, Codex, OpenAI, Anthropic, bots, or other development tools as commit authors, co-authors, contributors, or trailers.

Do not add Co-authored-by lines for tools. Do not add bot accounts to commits. Use the repository's already configured Git user.name and user.email for commits; do not modify identity to impersonate anyone.

GitHub contribution attribution is controlled by commit metadata and GitHub account/email matching. The project should not deliberately add tool identities to that metadata. If an external platform independently adds attribution that cannot be controlled from the repository, do not falsify metadata to bypass platform rules.

10. AUTOMATIC UPDATE AFTER LATER USER CHANGES

Whenever a future Claude Code/Codex session is started on this project, first detect local changes, untracked files, and remote changes. Preserve the user's edits, validate them where possible, integrate them with the current project, update affected tests/results/documentation as needed, then commit and push the validated changes if GitHub access is available.

Do not assume continuous background monitoring when no coding session/process is running. The automatic behavior applies whenever the project is opened/run by the coding tool. If the environment supports a safe persistent watcher explicitly requested by the user, it may be configured separately; otherwise never pretend the repository can update itself while no process is active.

11. CODE QUALITY

Write ordinary, readable, maintainable Python. Prefer straightforward functions and small coherent modules over unnecessary abstractions. Use classes only where state/reuse justifies them. Use type hints and docstrings where helpful. Comments should explain research assumptions, non-obvious mathematics, missingness mechanisms, leakage prevention, or important decisions - not obvious syntax.

Do not deliberately introduce fake mistakes, poor formatting, inefficiency, spelling errors, or artificial imperfections. The code must be understandable and defensible during a faculty demonstration.

12. RECOMMENDED STRUCTURE

Use approximately:
healthcare-data-imputation/
README.md
requirements.txt
.gitignore
data/raw/
data/processed/
notebooks/01_dataset_analysis.ipynb
notebooks/02_missing_data_analysis.ipynb
notebooks/03_experiment_analysis.ipynb
src/config.py
src/data_loader.py
src/preprocessing.py
src/missingness.py
src/imputation.py
src/genetic_selection.py
src/prediction.py
src/evaluation.py
src/visualization.py
experiments/run_baselines.py
experiments/run_imputation.py
experiments/run_ga.py
experiments/run_all.py
results/metrics/
results/figures/
results/tables/
results/logs/
tests/
docs/methodology.md
docs/dataset_selection.md
docs/literature_review.md
docs/experiment_log.md
docs/research_question_analysis.md
docs/results_summary.md
docs/progress_report.md
docs/supervisor_progress_summary.md
report_data/
Adapt rather than restructure a good existing repository.

13. DATASET DISCOVERY AND COLLECTION

Do not require the user to select or collect the primary dataset manually. Search reputable public sources such as UCI Machine Learning Repository, OpenML, Kaggle public datasets, PhysioNet when access conditions permit, government/open-health repositories, and established research repositories.

Identify approximately 3-5 strong candidates. For each collect: name, official/source URL, healthcare task, rows, features, target, feature types, existing missingness, class balance, license/access conditions, computational needs, literature availability, advantages, limitations, suitability for controlled MCAR/MAR/MNAR experiments, and suitability for GA feature selection.

Create results/dataset_candidate_comparison.csv and docs/dataset_selection.md.

Rank candidates using healthcare relevance, missing-data suitability, sample size, feature diversity, target clarity, class balance, reproducibility, public availability, computational feasibility, privacy/licensing, literature availability, and GA suitability.

Select ONE primary dataset before main experiments. Do not dataset-shop for favorable results.

14. DATA ACQUISITION, SAMPLES, AND PROVENANCE

Automatically download/acquire the selected public dataset when technically and legally possible. Store untouched source data in data/raw/ and derived data in data/processed/.

Collect the actual dataset records/data samples required for experiments, not merely links or descriptions. Preserve enough representative samples for smoke tests where licensing permits. Never fabricate data samples.

Create metadata containing dataset name, official/source URL, access date, version if known, license, citation, original filename, target, and checksum/hash.

If automated acquisition is impossible, produce the exact minimal user step required; do not ask the user to independently research the source.

Do not commit restricted data or data whose license forbids redistribution. If raw data should not be committed, add an acquisition script and provenance metadata so the dataset can be reproduced.

15. DATASET INTEGRITY AND ANALYSIS

After acquisition: calculate hash, verify readability, record shape/columns/dtypes, duplicates, missing markers, invalid/infinite values, class distribution, identifiers, leakage-prone columns, numerical/categorical features, descriptive statistics, suspicious values, and naturally missing values.

Never modify raw data in place. Fail clearly if the raw dataset unexpectedly changes.

16. EDA

Generate analytically useful plots: target/class distribution, feature distributions, missing-values-per-feature, missingness heatmap, correlation matrix when appropriate, and other justified summaries. Use clear titles, labels, legends, and report-quality resolution. Avoid decorative plots without analytical value.

17. DATA LEAKAGE PREVENTION

Mandatory: split before fitting imputers/scalers/encoders/feature selection. Never use test labels during preprocessing decisions, GA optimization, hyperparameter tuning, or method selection.

```
Conceptual flow:
Raw/Clean Data -> Train/Test Split -> Training-side preprocessing -> Controlled masking -> Fit imputer using train
```

18. TRAIN/TEST AND CV

For classification use stratification where possible. Start with 80% train / 20% held-out test. Use Stratified 5-fold CV inside training where appropriate. The held-out test set must not participate in GA fitness, feature selection, imputer/model selection, or hyperparameter tuning.

19. NATURAL VS SYNTHETIC MISSINGNESS

Distinguish naturally missing values from intentionally masked values. Ground truth is unknown for natural missingness; never claim reconstruction accuracy there. For controlled experiments, preserve original observed values before masking and calculate imputation error only on intentionally hidden cells.

20. MISSINGNESS MECHANISMS

Support approximately 10%, 20%, and 30% missingness.

MCAR: mask eligible cells independently at random.
MAR: missingness probability depends on one or more observed variables; document conditioning and affected features and probability construction.
MNAR: missingness depends on the value being masked or another defensible approximation; document the exact assumption.

Never mask target, identifiers, or protected columns. Record requested and actual missing percentages.

21. IMPUTATION METHODS

Implement:
A. Mean imputation for suitable numerical attributes.
B. Median imputation.
C. KNN imputation.
D. One manageable ML-based method such as IterativeImputer or a defensible Random-Forest-style iterative/MissForest-like method.
E. Optional Autoencoder only after the classical/ML/GA pipeline is working and only if scientifically justified.

Do not add methods merely to inflate scope.

22. IMPUTATION EVALUATION

For intentionally masked numerical values calculate MAE and RMSE; use R2 only when appropriate. For categorical reconstruction use suitable categorical metrics if needed. Record metrics by method, mechanism, missingness percentage, seed, and optionally feature.

23. RANDOM FOREST PREDICTION

Use Random Forest as the primary downstream prediction model. Establish a simple baseline first. Tune only through training/CV when justified. Record parameters, split, seed, CV metrics, test metrics, and runtime. Never tune against held-out test data.

24. PREDICTION METRICS

Calculate Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC for binary classification where valid. Optionally PR-AUC when class imbalance makes it useful. Explicitly define averaging for multiclass tasks and do not silently suppress undefined metrics.

25. GENETIC ALGORITHM FEATURE SELECTION

Represent a feature subset as a binary chromosome; 1=selected, 0=removed. Prevent zero-feature chromosomes.

Configurable starting defaults:
population_size=30
generations=30
crossover_probability=0.8
tournament_size=3
elite_count=2
mutation_probability approximately 1/number_of_features

Use understandable selection/crossover/mutation/elitism. Fitness must use training data/CV only. Prefer mean CV F1 where appropriate, optionally with a small feature-count penalty:
fitness = mean_CV_F1 - lambda * selected_feature_ratio.
Document lambda and all GA parameters.

26. GA LOGGING AND STABILITY

Record each generation's best/mean fitness, selected-feature count, and best chromosome. Generate convergence plots.

Across repeated runs calculate selection frequency per feature, average selected feature count, variability, frequently selected subset, and unstable features. Create results/tables/ga_feature_frequency.csv and a corresponding figure.

27. ALL FEATURES VS GA

For each relevant imputed dataset compare Random Forest using all features against the same model using GA-selected features on the same split. Report feature count before/after, percentage reduction, Accuracy, Precision, Recall, F1, ROC-AUC when applicable, and runtime.

28. EXPERIMENT GROUPS

E0 Complete/reference-data RF baseline.
E1 Mean/Median imputation.
E2 KNN imputation.
E3 ML-based imputation.
E4 Optional Autoencoder if justified.
E5 GA feature selection.
E6 Imputation + GA + Random Forest.
E7 Comparison with selected existing approaches/literature where methodologically valid.

29. EXPERIMENT MATRIX AND REPEATS

Automatically build combinations of missingness mechanism x missingness percentage x imputation method x GA state x random seed. Use clear IDs such as MCAR_10_KNN_GA_RF_SEED42.

Use several seeds when practical, initially 42, 123, 2026, 7, 99. Store individual runs and aggregate mean +/- standard deviation. Create results/experiment_manifest.csv.

30. AUTOMATIC EXPERIMENT EXECUTION

Provide a master command such as:
python experiments/run_all.py

It should validate the dataset, build the manifest, identify completed runs, execute missing runs, save each run immediately, log failures, continue independent experiments after recoverable failures, aggregate results, generate tables/figures, update research documentation, and support checkpoint/resume without rerunning valid completed experiments.

31. RESULT STORAGE

Every run must produce structured CSV/JSON data including experiment ID, timestamp, dataset/hash, mechanism, requested/actual missingness, imputation method, seed, GA state, selected features/count, model parameters, imputation metrics, classification metrics, runtime, success/failure, and failure reason.

Never silently discard failures or replace failed values with zero.

32. AGGREGATION, RANKING, ABLATION

Aggregate by mechanism x missingness x imputation x GA state. Calculate mean, standard deviation, min, max, and successful-run count.

Identify lowest MAE/RMSE, highest F1/ROC-AUC where appropriate, robustness as missingness increases, GA/no-GA tradeoffs, and smallest competitive feature subset. Do not define 'best' from one metric alone.

Where feasible compare:
complete + RF
imputed + RF
complete + GA + RF
imputed + GA + RF.

33. STATISTICAL ANALYSIS

When repeated paired runs support it, use a justified paired test such as paired t-test or Wilcoxon signed-rank test. Record hypotheses, sample size, test, statistic, p-value, interpretation, limitations, and multiple-comparison correction where genuinely necessary. Do not add tests for appearance.

34. FAILURE AND NEGATIVE-RESULT ANALYSIS

Analyze scientifically useful failures: deterioration at high missingness, GA reducing performance, unstable feature selection, MNAR difficulty, weak Autoencoder performance, class-specific degradation, or runtime tradeoffs. Retain negative findings.

35. LITERATURE DISCOVERY

Search focused literature on healthcare data imputation, MCAR/MAR/MNAR, Mean/Median/KNN/iterative/ML imputation, Random Forest healthcare prediction, GA feature selection, healthcare feature selection, and imputation+feature-selection pipelines.

Prioritize peer-reviewed journals, reputable conferences, systematic reviews, foundational work, and recent relevant studies. Target roughly 15-25 strong references, not hundreds of weak ones.

36. LITERATURE VERIFICATION AND MATRIX

Never invent references. Verify title, authors, year, venue, DOI and publisher/source where possible.

Create results/literature_matrix.csv with paper, authors, year, dataset, healthcare task, mechanism, missingness level, imputation method, feature selection, prediction model, metrics, findings, limitations, DOI/URL, and relevance. Create docs/literature_review.md.

For literature-motivated baselines, document whether the implementation is exact or adapted. Never present results from different datasets as direct head-to-head comparisons.

37. REQUIRED TABLES AND FIGURES

Generate report-ready tables for dataset candidates, selected dataset characteristics, natural/controlled missingness, imputation MAE/RMSE, prediction before GA, selected features, feature stability, prediction after GA, all-features vs GA, statistical comparisons, overall ranking, and literature comparison.

Generate figures for research workflow, class distribution, missingness, MAE/RMSE/F1 vs missingness, ROC-AUC where relevant, imputer comparison, GA convergence, feature-selection frequency, all-vs-selected performance, mechanism comparison, and runtime where useful.

Export CSV plus Markdown/LaTeX-ready tables where useful and high-resolution PNG/PDF figures.

38. RESEARCH QUESTION ANALYSIS

Create docs/research_question_analysis.md and answer RQ1-RQ4 only from actual experiment evidence. Analyze trends across 10/20/30%, MCAR/MAR/MNAR, reconstruction quality vs downstream prediction, GA feature reduction vs performance, stability, and contextual literature comparisons.

39. CONFIGURATION, REPRODUCIBILITY, TESTING

Centralize dataset path, target, protected columns, seeds, split ratio, mechanisms, missingness levels, imputers, GA settings, RF settings, and output paths.

Provide documented Python version, requirements, deterministic seeds where supported, dataset checksum, experiment manifest, one-command full execution, and smaller debug commands.

Test dataset loading, target detection, missingness rates, target preservation, deterministic masking, dataframe immutability, imputation shape/completeness, GA validity/non-empty subset, RF execution, serialization, manifest/checkpoint behavior, and a small end-to-end pipeline.

40. PERFORMANCE, SECURITY, ETHICS

Prefer ordinary workstation execution. Avoid unnecessary GPU/distributed infrastructure and huge searches. Add progress indicators and safe caching.

Use public, appropriately licensed/de-identified healthcare data. Never commit PII, passwords, tokens, API keys, or credentials. Do not make clinical claims beyond the evidence. This is a research experiment, not a medical decision system.

41. PROGRESS REPORT FOR FACULTY SUPERVISOR - MANDATORY

Maintain docs/progress_report.md after every major phase. It must include:
- project title
- objective
- research questions
- work completed
- current dataset and characteristics
- methodology
- experiments completed
- actual current results
- GA status
- important tables/figures
- problems/limitations
- work in progress
- next steps.

Also maintain docs/supervisor_progress_summary.md as a concise faculty-facing report with topic, problem, objective, dataset and selection rationale, methodology, completed work, current experiments/results, important graph/table references, GA-selected features when available, issues, deviations from plan, and immediate next work.

Distinguish IMPLEMENTED vs EXECUTED vs VALIDATED. Never fabricate progress.

42. AUTOMATIC SUPERVISOR REPORT EXPORT

After each substantial milestone, generate a clean supervisor-ready PDF report in addition to Markdown when the environment has a reliable PDF generation path.

Preferred output:
reports/RP_Progress_Report.pdf

The PDF should be readable without inspecting source code and should contain only verified current progress. Include selected report-ready figures/tables when available. If PDF generation dependencies are unavailable, keep the Markdown/LaTeX source ready and record the blocker; do not invent results.

43. FINAL REPORT PACKAGE

Maintain report_data/ with dataset_summary.md, methodology_summary.md, experiment_setup.md, results_summary.md, research_questions.md, discussion_points.md, limitations.md, future_work.md, references.bib, tables/, and figures/.

After experiments are complete, prepare a full academic report draft: Abstract; Introduction; Background; Problem Statement; Objectives; Research Questions; Literature Review; Dataset; Preprocessing; Missing-Data Mechanisms; Methodology; Imputation Methods; GA Feature Selection; Random Forest; Experimental Setup; Metrics; Results; Statistical Analysis; Discussion; Existing-Work Comparison; Limitations; Future Work; Conclusion; References.

Own results and published results must be clearly separated.

44. PHASED EXECUTION

```
Proceed automatically through:
1 Repository/environment audit
2 Dataset discovery
3 Candidate comparison
4 Primary dataset selection
5 Acquisition/integrity
6 Preprocessing/EDA
7 Complete-data RF baseline
8 Missingness framework
9 MCAR 10/20/30
10 Mean/Median
11 KNN
12 ML imputation
13 RF after imputation
14 GA implementation
15 GA baseline
16 Imputation+GA+RF
17 MAR
18 MNAR
19 Repeated seeds
20 Feature stability
21 Ablation
22 Statistical comparison
23 Literature comparison
24 Tables
25 Figures
26 RQ analysis
27 Optional Autoencoder if justified
28 Final experiment audit
29 Progress documentation
30 Report data package
31 Final report draft
32 Final reproducibility/repository audit.

After every validated phase: test -> save -> document -> commit -> push -> continue, when GitHub access permits.
```

45. CONTINUOUS EXECUTION AND STOP CONDITIONS

Do not ask 'Should I continue?' between routine phases. Continue automatically.

Stop only for restricted credentials/access, legal/ethical approval, a faculty decision materially changing scope, equally valid incompatible research directions, a fundamental environment blocker, or repeated failed debugging where user action is genuinely required.

When blocked, state the blocker, attempts made, exact minimal user action, and the next command/action.

46. FUTURE PROJECT UPDATES

Whenever the user later changes source code, configuration, notebooks, documentation, or data setup and starts Claude Code/Codex again:
- inspect the new local state
- preserve the user's changes
- fetch remote changes safely
- determine affected pipeline components
- run targeted tests/experiments
- update dependent results/docs only when necessary
- commit the validated update
- push to GitHub if access permits.

Never overwrite user changes merely to restore a previous generated version.

47. FINAL AUDITS

Before declaring completion verify:
DATA: provenance, license, citation, raw preservation, checksum, target, identifiers, privacy.
METHODOLOGY: no leakage, correct masking, correct imputation fitting, target protected, GA training/CV only.
EXPERIMENTS: manifest, seeds, individual results, repeats, failures, aggregation, checkpointing.
GA: valid chromosomes, parameters/fitness documented, convergence, feature names, stability.
STATISTICS: correct summaries/tests/sample sizes/p-values.
LITERATURE: verified references, no fabricated citations, cross-dataset comparisons labeled.
REPORT: abstract/methodology/results/tables/figures match actual artifacts; limitations and negative findings retained.
CODE: imports/tests/master command, dependencies, no secrets, no unwanted tool-specific files, valid README commands.
GIT/GITHUB: intended changes committed, no prohibited files/secrets, remote push verified where permitted.

48. PRIMARY SUCCESS CRITERIA

Success means: defensible dataset selection; correct missingness simulation; multiple imputers evaluated; correct reconstruction error; downstream prediction evaluated; GA implemented without leakage; repeated experiments; statistical summaries; evidence-based RQ answers; reproducibility; honest limitations; auditable result trail; current supervisor progress report; final report draft; synchronized GitHub repository where access permits.

High accuracy alone is not success.

49. FINAL PRINCIPLE

Do not force the proposed method to win. If Mean, Median, KNN, ML imputation, no-GA, or GA performs best, report what the evidence shows. If GA mainly reduces feature count while preserving performance, treat that as a potentially useful finding. If mechanisms favor different methods, analyze the pattern.

The goal is a defensible research result, not a predetermined conclusion.

50. START NOW

Inspect the actual repository and environment. Do not merely describe intended work.

```
If no dataset is selected: discover -> shortlist -> rank -> select -> acquire -> validate -> analyze.
```

Then automatically execute the remaining phases, validate each one, save actual outputs, update progress documentation, create/update the supervisor report, commit and push each validated phase when GitHub access is available, and continue until the achievable RP pipeline, analysis, documentation, and final report draft are complete.

Use available terminal, internet, package manager, Python, notebook, test, and version-control capabilities where permitted.

Only involve the user when genuinely necessary.

START.